ESP32-Based Intelligent Battery Management System (BMS)

Complete Internship Project Report

Student name	Harsh Rajesh Sonkamble
Institute / department	Vidhyapratistan Kamal Nayan Bajaj Institue Of Engineering & Technology . Department = E&tc
Internship organization	Eleven Skills
Submission date	10th November 2026
Domain	Embedded Systems / Internet of Things (IoT)
Controller	ESP32 microcontroller
Simulation	Wokwi
Cloud platform	Blynk IoT
Project scope	Integrated Tasks 1–6

Prepared as a technical record of the design, implementation and simulation of a four-cell battery monitoring and protection system. Fill in the student/institution details and insert the final public project links before submission.

Declaration and Project Details

This report describes an ESP32-based Battery Management System developed as an integrated embedded/IoT internship project. The implementation is demonstrated in the Wokwi simulator and uses the Blynk IoT platform for cloud monitoring. The report records implemented features and tests reported during development; it does not claim physical battery-pack certification or laboratory validation.

Project links to complete before submission

Item	Link / details
GitHub repository	PASTE PUBLIC GITHUB URL HERE
Wokwi project	PASTE PUBLIC WOKWI URL HERE
Blynk dashboard evidence	See screenshots folder / insert evidence link
Demonstration video	PASTE VIDEO LINK OR FILE LOCATION HERE

Abstract

The project implements a four-cell battery monitoring and protection system using an ESP32. It samples individual cell voltages through ADC inputs, calculates pack-level parameters, identifies cell imbalance and evaluates battery health. A safety layer controls a relay and buzzer in response to unsafe conditions. A 16×2 I2C LCD provides local status information. A fault-tolerant runtime layer classifies operating conditions into NORMAL, DEGRADED, FAILSAFE and SHUTDOWN modes and records diagnostic events. Blynk IoT is used to publish telemetry, diagnostic state, risk level and operator recommendations. The dashboard presents live values and historical voltage trends. The system is developed and evaluated in simulation; actual hardware deployment requires calibrated measurement circuitry and additional safety validation.

Table of Contents

• 1. Introduction
• 2. Objectives and Scope
• 3. Requirements and Tools
• 4. System Architecture
• 5. System Workflow
• 6. Task-wise Implementation
• 7. Testing and Results
• 8. Challenges and Solutions
• 9. Limitations and Future Scope
• 10. Conclusion
• Appendix A – Key Configuration Summary
• Appendix B – Final Evidence Checklist

1. Introduction

A Battery Management System monitors battery operating conditions and supports safe use by observing cell voltages, detecting abnormal conditions and controlling protective actions. In a multi-cell pack, individual cell measurements are important because the total pack voltage alone can hide a weak or overcharged cell. This project demonstrates a modular BMS using an ESP32, Wokwi and Blynk.

The project is organized as a single integrated Wokwi design. Each task adds a layer: cell intelligence, protection, human-machine interface, fault tolerance, cloud telemetry and an executive dashboard.

2. Objectives and Scope

- Measure four simulated cell voltages independently using ESP32 ADC channels.
- Calculate pack voltage, average cell voltage, highest and lowest cells, and cell imbalance.
- Classify battery condition and trigger protective relay/buzzer behavior when required.
- Display key measurements, status and diagnostic messages on an I2C LCD.
- Detect sensor disconnection, invalid values, potentially frozen ADC readings and relay feedback mismatch.
- Support runtime modes NORMAL, DEGRADED, FAILSAFE and SHUTDOWN, with fault logging and recovery handling.
- Send telemetry and fault-related information to Blynk, including after connectivity is restored.
- Provide live dashboard values, historical charts, risk-level indication and operator recommendations.

Scope boundary

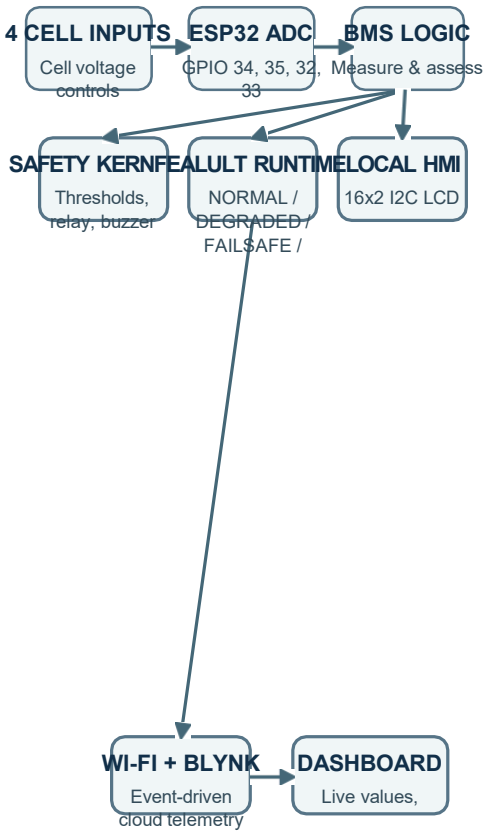
The project is a simulation-oriented educational prototype. It does not include a certified battery monitor IC, cell balancing hardware, current measurement, temperature sensing, contactor safety certification or a validated high-voltage design. The voltage-scaling method used in simulation must be recalculated and calibrated for any physical circuit.

3. Requirements and Tools

Category	Components / tools
Controller	ESP32 development board in Wokwi
Cell inputs	Four analog input channels: GPIO 34, 35, 32 and 33
Protection outputs	Relay control GPIO 25; buzzer GPIO 26
Feedback input	Relay feedback GPIO 27 (requires a correctly wired feedback signal)
Display	16×2 I2C LCD, address 0x27; SDA GPIO 21, SCL GPIO 22
Firmware	Arduino C++ using ESP32 Arduino core
Libraries	WiFi, BlynkSimpleEsp32, Wire, LiquidCrystal_I2C, math
Cloud	Blynk IoT template and virtual datastreams
Simulation	Wokwi

4. System Architecture

The architecture separates sensing, decision-making, local protection, diagnostics and cloud presentation. Cell inputs are sampled by the ESP32. Battery calculations and fault checks feed the runtime mode and protection logic. The LCD provides local feedback, while Wi-Fi and Blynk deliver telemetry to the remote dashboard.



chCarotsn,creispktu&al system architecture – confirm final wiring against the Wokwi circuit.
advice

Figure 1. Conceptual architecture of the integrated ESP32 BMS.

5. System Workflow

The controller initializes the hardware and communication interfaces, then repeatedly samples the four cell inputs. It validates the readings, computes battery metrics, updates the system mode and applies the appropriate protection and reporting actions. Periodic work is scheduled using non-blocking timing so that the firmware can continue servicing communication and monitoring tasks.

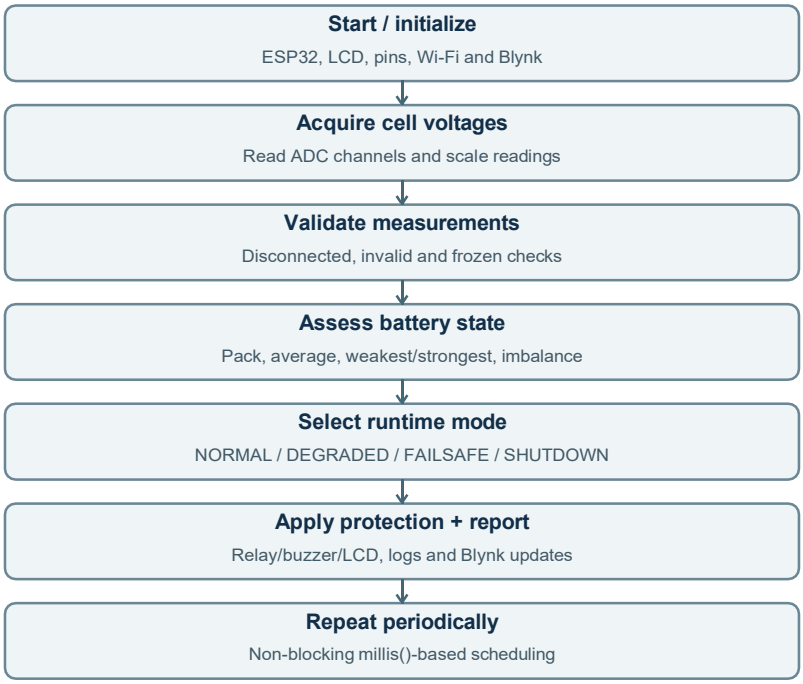


Figure 2. Main firmware workflow. The actual sketch loop may contain additional helper calls and scheduling conditions.

6. Task-wise Implementation

Task 1 – Adaptive Multi-Cell Battery Intelligence Engine

Four cell channels are read independently. The firmware converts raw ADC readings into estimated cell voltages using the configured scaling factor. It calculates pack voltage as the sum of the cell estimates and average voltage as pack voltage divided by four. It also tracks the highest and lowest cell and calculates the voltage difference (imbalance). Battery health is classified using the configured thresholds and sensor conditions.

Key data includes NUM_CELLS = 4, cellPins = {34, 35, 32, 33}, raw ADC values, cellVoltage[], packVoltage, averageVoltage, highest/lowest cell and imbalance. The simulated conversion uses a 12-bit ADC range (0–4095) and an assumed 3.3 V reference; physical accuracy requires calibration and an appropriate input divider/protection network.

Task 2 – Event-Driven Safety Protection Kernel

The safety layer evaluates cell voltage limits and battery health, then commands the protection relay and buzzer when unsafe conditions are detected. The implementation uses millis()-based timing rather than delay()-based blocking waits. This allows monitoring and communication tasks to continue while timers manage event handling and anti-chatter behavior. Relay cutoff is used as the protective response in the simulation.

Task 3 – HMI and Diagnostics

A 16×2 I2C LCD is used as the local human-machine interface. The firmware updates the display with battery measurements, operating state and diagnostic information. The screen update routine is scheduled so the display can rotate or refresh without blocking the monitoring loop. The LCD uses I2C address 0x27 in the current project configuration.

Task 4 – Fault-Tolerant Embedded Runtime System

The runtime layer detects sensor disconnection, invalid readings and potentially frozen ADC channels. It maintains per-cell fault flags and uses a consecutive-sample counter for frozen-reading detection, including a check that other channels have changed before treating one channel as potentially frozen. Faults are logged with timestamps and recovery messages. The system mode is selected from NORMAL, DEGRADED, FAILSAFE and SHUTDOWN. Relay mismatch monitoring compares the commanded relay state with the feedback input after a configured delay.

Observed validation included disconnected-cell behavior leading to FAILSAFE and relay OFF, followed by healthy readings recovering to NORMAL. Frozen ADC detection and recovery logic were exercised during development. Relay mismatch detection is implemented in the sketch but its end-to-end simulation test remains to be confirmed; the feedback pin must be wired to a meaningful simulated feedback signal before claiming this test passed.

Task 5 – Intelligent Cloud Telemetry Architecture

The ESP32 connects to the Wokwi guest Wi-Fi network and communicates with Blynk. Telemetry is designed to be event/state-change oriented rather than relying only on a fixed stream of unchanged values. Reconnection handling periodically checks Wi-Fi and Blynk availability. A fault event queue is used to retain events while the cloud connection is unavailable and synchronize them after reconnection. Signal quality is monitored using Wi-Fi RSSI.

Task 6 – Executive Battery Intelligence Dashboard

The Blynk web dashboard displays individual cell voltage, pack voltage, average voltage, imbalance, battery status, system mode, relay state, Wi-Fi signal quality, fault event history, risk level and operator recommendation. Historical charts are configured separately for each cell and for pack voltage. Separate cell charts were used because this presentation reliably displayed all four cell histories in the tested dashboard.

Risk reporting maps severe states such as FAILSAFE/SHUTDOWN or pack failure to FAILURE, lesser abnormal states to WARNING, and healthy operation to NORMAL. The recommendation logic provides messages such as inspecting sensors, inspecting relay feedback, inspecting the battery, balancing cells, monitoring imbalance or confirming normal operation. The tested critical imbalance case displayed FAILURE with “Critical: Inspect Battery”; the healthy recovery case displayed NORMAL with “System Normal”.

7. Testing and Results

The following results summarize tests reported during the Wokwi and Blynk development sessions. Values are illustrative observed simulation values, not laboratory measurements.

7.1 Healthy battery test

Parameter	Observed value
Cell 1	3.33 V
Cell 2	3.33 V
Cell 3	3.36 V
Cell 4	3.37 V
Pack voltage	13.39 V
Average voltage	3.35 V
Imbalance	0.045 V
Battery status / system mode	Healthy / NORMAL
Relay command / mismatch	ON / NO
Blynk risk / recommendation	NORMAL / System Normal

7.2 Critical imbalance test

Parameter	Observed value
Cell voltages	3.50 V, 3.33 V, 3.30 V, 3.37 V
Imbalance	0.206 V
Battery status / system mode	Critical Imbalance / FAILSAFE
Relay command	OFF
Blynk risk level	FAILURE
Operator recommendation	Critical: Inspect Battery

7.3 Sensor disconnection test

A cell input was adjusted to 0.00 V. The serial output reported the affected cell sensor as disconnected, the battery status became Pack Failure, the system entered FAILSAFE and the relay command changed to OFF. After restoring a healthy cell reading, the system was observed returning to NORMAL in a separate recovery test.

7.4 Dashboard and historical data tests

The user confirmed that live values for V0–V3 updated, individual historical charts showed each cell's readings, and a separate pack voltage chart displayed history after the configured aggregation interval. Risk level and operator recommendation were checked in both critical and recovered healthy conditions.

7.5 Verification status summary

Test / feature	Status based on reported evidence
Four-cell live telemetry	Verified
Pack and average calculations	Verified

Test / feature	Status based on reported evidence
Separate cell history charts	Verified
Pack history chart	Verified
Critical imbalance → FAILSAFE and relay OFF	Verified
Sensor disconnect detection	Verified
Healthy recovery → NORMAL	Verified
Blynk risk and recommendation changes	Verified
Frozen ADC detection / recovery	Exercised during development; retain evidence
Relay mismatch detection	Implemented; end-to-end test still to confirm
LCD display	Implemented; capture final screenshot

Before final submission, run and capture the remaining unverified checks, especially relay feedback mismatch and a final LCD demonstration. Update the table if additional tests are completed.

8. Challenges Faced and Solutions

- Compilation errors caused by mismatched braces or misplaced else statements were corrected by restructuring the affected functions.
- Zero or incorrect cell readings were investigated by checking the Wokwi input connections and ADC channel mapping.
- Frozen ADC detection required additional logic to avoid treating every stable battery reading as a fault; the implementation uses a counter and cross-channel activity check.
- Historical charts were difficult to interpret when multiple cell datastreams were combined. Separate charts for each cell provided a clear display.
- Cloud telemetry required reconnection handling and event synchronization so fault events could be delivered after connectivity returned.
- Dashboard risk and recommendation widgets were validated using critical imbalance and healthy recovery scenarios.

9. Limitations and Future Scope

Current limitations

- The implementation is validated in simulation and has not been certified for use with a real lithium battery pack.
- ADC scaling and reference voltage are simplified assumptions; actual ESP32 ADC characteristics and divider tolerances require calibration.
- No cell balancing circuit, pack current sensor, cell temperature sensor or hardware overcurrent protection is included in the current scope.
- Relay mismatch verification depends on a correctly connected feedback signal and should be demonstrated before submission.
- Cloud telemetry depends on Wi-Fi and Blynk service availability.

Future enhancements

- Add calibrated voltage sensing and per-cell temperature monitoring.
- Add current measurement, state-of-charge estimation and cell balancing.

- Use a dedicated battery-monitoring IC and appropriately rated protection hardware.
- Add persistent local fault storage and stronger event acknowledgement on cloud reconnection.
- Validate relay command/feedback behavior with a fully wired simulated or physical feedback circuit.
- Perform controlled hardware testing with current-limited supplies and appropriate safety procedures.

10. Conclusion

The ESP32-Based Intelligent Battery Management System demonstrates an integrated approach to four-cell battery monitoring, battery-state assessment, event-driven protection, local diagnostics, fault-tolerant runtime management and cloud-based visualization. The Wokwi simulation and Blynk dashboard provide a practical environment for demonstrating live telemetry, historical voltage trends, risk reporting and operator recommendations. Reported tests confirmed healthy operation, critical imbalance response, sensor disconnection handling and recovery. Remaining verification items are explicitly identified so that the final submission accurately reflects the demonstrated implementation.

Appendix A – Key Configuration Summary

Setting	Configured value
Cell count	4
ADC pins	34, 35, 32, 33
Relay control	GPIO 25
Buzzer	GPIO 26
Relay feedback	GPIO 27
LCD	16×2 I2C, address 0x27
LCD I2C pins	SDA 21, SCL 22
ADC nominal range	0–4095
ADC assumed reference	3.3 V
Cell overvoltage threshold	4.20 V
Cell undervoltage threshold	3.00 V
Disconnected voltage threshold	0.10 V
Invalid voltage threshold	4.50 V
Minor imbalance threshold	0.05 V
Critical imbalance threshold	0.15 V
Frozen ADC limit	10 consecutive checks
ADC change threshold	5 raw counts
Relay mismatch delay	500 ms

Appendix B – Final Evidence Checklist

- Insert the public GitHub repository URL and verify it opens without login.
- Insert the public Wokwi project URL and verify the project opens without access approval.
- Add a screenshot of the complete Wokwi circuit and the Serial Monitor healthy-state output.

- Add evidence of sensor disconnect, critical imbalance and recovery behavior.
- Add screenshots of all four individual cell historical charts and the pack-voltage chart.
- Add screenshots of Blynk NORMAL and FAILURE risk states with recommendations.
- Capture the LCD screen and complete the relay feedback mismatch test.
- Record a clear 2–5 minute demonstration video and check audio, readability and file size (maximum 100 MB).
- Review README, Bug Log, Assumptions and Self-Attestation files for accuracy.
- Remove all private credentials, including Blynk authentication tokens, from public source code.

End of Report